



BLoC Pattern — Complete Guide

BLoC (Business Logic Component) is a state management pattern created by Google engineers in 2018. It enforces a strict, predictable architecture for managing state in Flutter applications, separating business logic from UI completely.

The Problem With setState()



Too Primitive

Business logic lives directly inside widgets, making it impossible to share state between screens or test without a widget context.

- Logic mixed with UI code
- Can't share state across screens
- Untestable without widget tree
- Rebuilds entire widget tree

Provider: Logic Still Leaks

Widgets Become Decision Makers

Complex logic piles up in widgets – conditions, validations, and async chains bleed into UI code.

No Clear Separation

Hard to distinguish between UI rendering and business logic. Testing button logic becomes difficult.

Duplicated Logic

Same complex logic appears across multiple screens, violating DRY principles and creating maintenance nightmares.



Riverpod: Tools Without Structure

Riverpod provides powerful tools but no enforced architecture pattern. Different developers write logic differently, leading to inconsistent codebases.

1

Developer A

StateNotifierProvider with custom
notifier

2

Developer B

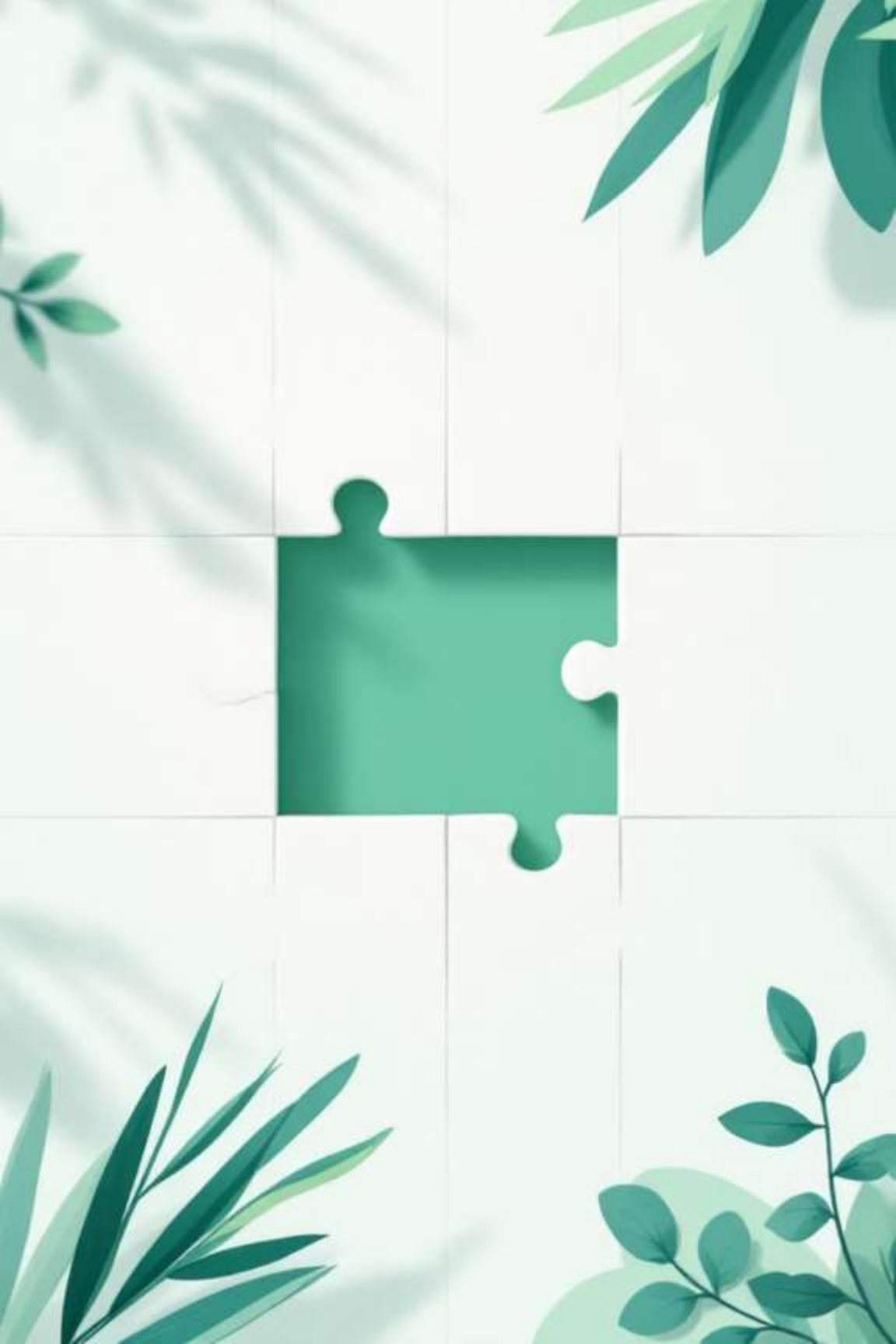
FutureProvider with async loading

3

Developer C

ChangeNotifierProvider approach

- ❏ No standard way to handle loading states, error states, sequential async events, or complex user flows across teams.

A decorative background on the left side of the slide. It features a light green grid pattern. In the center of the grid, there is a dark green puzzle piece that is missing from its position. Surrounding the grid are various green leafy branches, some in the top left, some in the top right, and some in the bottom left and bottom right corners.

What All These Lacked

01

User taps button

02

Where does logic go?

03

How does state flow?

04

What triggers rebuild?

Nobody enforced WHERE logic lives, HOW state flows, or WHAT triggers a rebuild. BLoC was created to answer these questions with a strict, predictable pattern.

What is BLoC?

One-Way Data Flow

BLoC enforces a unidirectional flow using Events and States with Streams. The UI only sends events and reacts to states – it never makes decisions.



Events

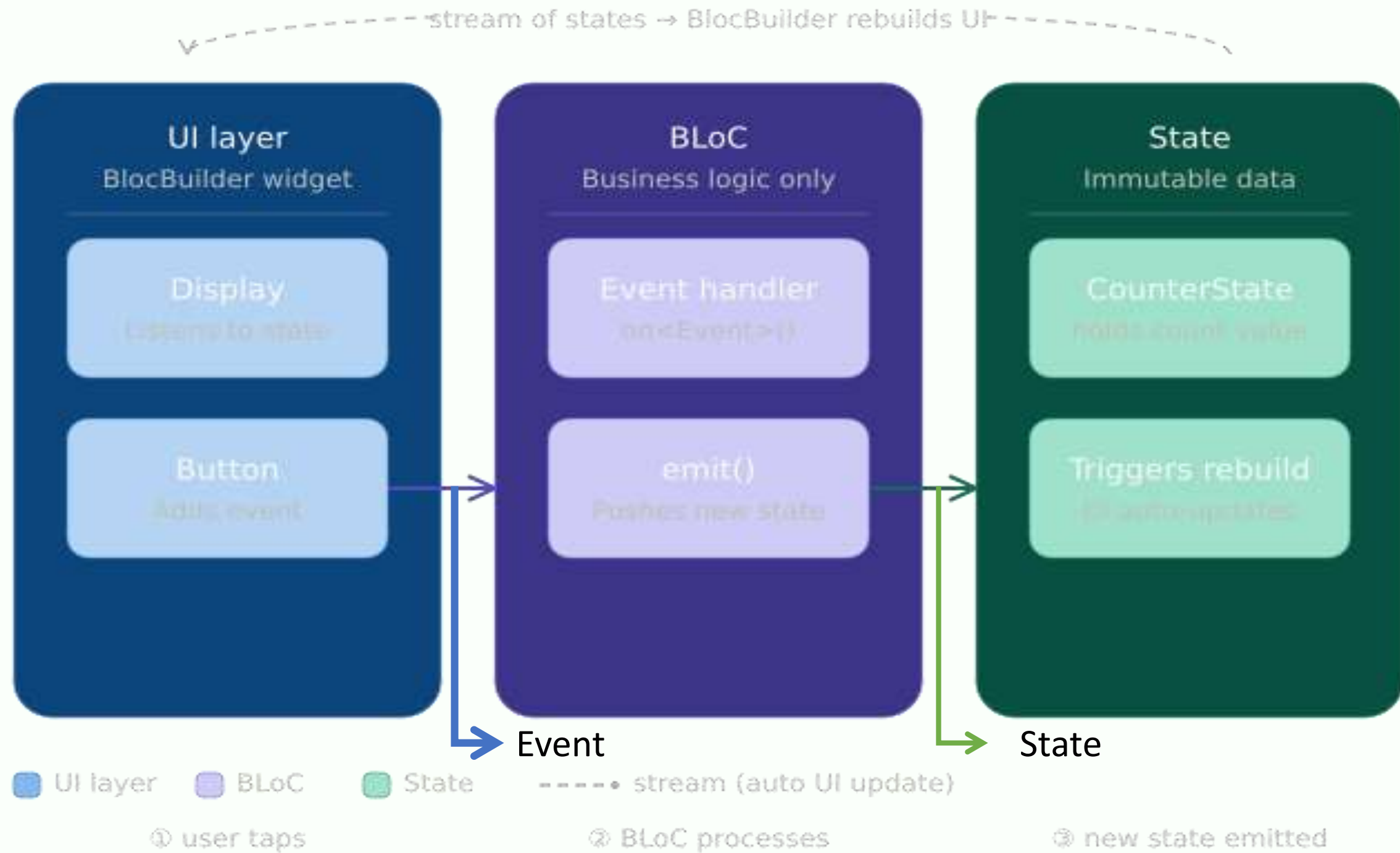
Things that happen – user actions, lifecycle triggers, network responses

BLoC

The brain – receives events, runs logic, emits states

States

Things that change – what the UI should show at any moment





When to Use BLoC

1.

Large Teams

Multiple developers need consistent patterns and enforced architecture

2.

Complex Flows

Many states and user interactions require strict separation

3.

Testability

Excellent test coverage with pure Dart tests

4.

Enterprise Apps

Scalable architecture for large, maintainable codebases



Skip BLoC for: Small apps or prototypes, solo developers, simple state needs (counter, theme toggle), or when time is limited – boilerplate is heavy.